water allocation

Purpose

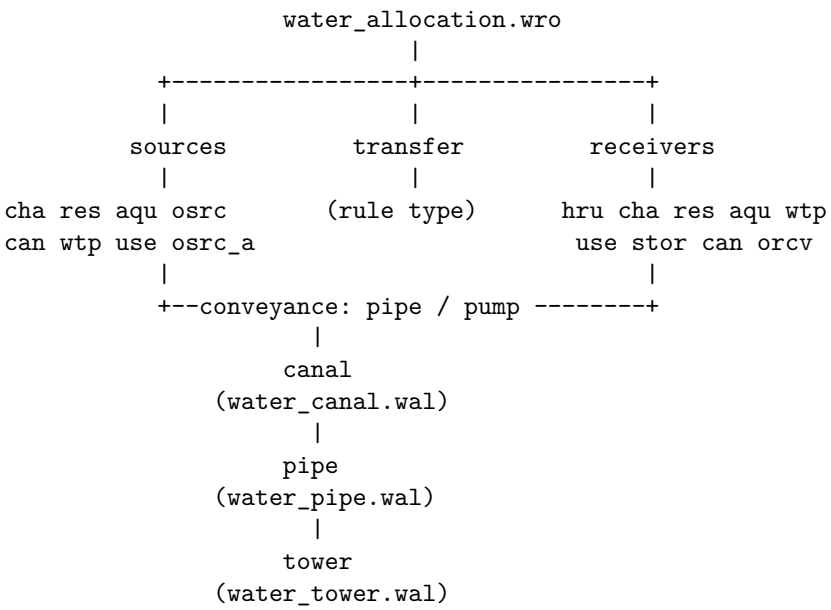
The water allocation module moves water between spatial objects (channels, reservoirs, aquifers, HRUs) and an auxiliary transport network (canals, pipes, towers, treatment plants, end-use sinks, out-of-basin sources and receivers). It is the mechanism SWAT+ uses for irrigation withdrawals, municipal and industrial supply, inter-basin transfer, and any rule-driven diversion. The driver type is `water_allocation` (`type water_allocation`, alias `wallo`) defined in `water_allocation_module.f90`. The entry-point file is `water_allocation.wro`. All other water-allocation files supply objects referenced from that file.

How it fits together

A single water allocation object (`wallo(iwro)`) owns one or more transfer objects (`wallo%trn(itrn)`). Each transfer object has:

- a transfer type (`trn_typ`): `outflo`, `ave_day`, `div_min`, `div_frac`, `dtbl_con`, `dtbl_lum`. The first five draw from the source list directly; `dtbl_lum` and `dtbl_con` defer the amount to a decision table.
- one or more source objects (`src`), each with a type from `cha`, `res`, `aqu`, `osrc`, `osrc_a`, `wtp`, `use`, `can`, plus a conveyance pointer (`pipe` or `pump`) and an optional withdrawal-limit decision table.
- exactly one receiving object (`rcv`), with a type from `hru`, `cha`, `res`, `aqu`, `wtp`, `use`, `stor` (tower), `can`, `orcv`.

The transport network is a set of independent files, each read by its own subroutine:




```
wtp  : water_treat.wal     (treatment plant; lag-and-loss store)
use  : water_use.wal       (domestic/industrial/commercial sink)
osrc : out_src.wal         (out-of-basin source, recall-driven)
orcv : outside_rcv.wal     (out-of-basin receiver)
```

The water allocation routine runs per day. `wallo_control` advances the current transfer object, computes demand (`wallo_demand`), withdraws from each source in turn (`wallo_withdraw`), allows compensation between sources that

share a `comp = y` flag, transports water through any pipe/pump conveyance (`wallo_transfer`), and delivers it to the receiver. Receiver-side post-processing (`wallo_treatment`, `wallo_use`, `wallo_canal`) handles lag, loss and outflow for `wtp`, `use` and `can` receivers.

Activation

The water allocation files are read at startup from `main.f90`, in this order:

```
call om_treat_read
call om_use_read
call om_osrc_read
call water_treatment_read
call water_use_read
!call water_osrc_read
call water_tower_read
call water_pipe_read
call water_canal_read
call water_allocation_read
```

The module is active whenever `water_allocation.wro` exists and is not `null`. The `wallo` array is then allocated from the count on line 2. If the file is absent the array is allocated as `wallo(0:0)` and the module is inert.

Per-step execution of `wallo_control` is triggered from `command.f90` (channel and reservoir commands) and from `sd_channel_control3.f90` (channel routing). The check `wallo(iwallo)%trn_cur <= wallo(iwallo)%trn_obs` advances the current transfer object until all are processed for the day.

Output (`water_allo_*.txt` / `.csv`) is produced when `db_mx%wallo_db > 0` and the corresponding `pco%water_allo` switch in `print.prt` is `y`.

Files

`water_allocation.wro`

Master file. Reader: src/water_allocation_read.f90. Pointed to by `in_watrts%transfer_wro` in `input_file_module.f90`.

Layout:

- Line 1: title (skipped).
- Line 2: `imax`, the number of water allocation objects. Allocates `wallo(imax)`.
- For each allocation object:
  - Header line (skipped).
  - One record with the object header: `name`, `rule_typ`, `trn_obs`.
  - Header line (skipped).
  - `trn_obs` transfer-object records.

Allocation-object header columns (`wallo(iwro)`):

#	Field	Type	Description
1	name	char(25)	name of the water allocation object
2	rule_typ	char(25)	rule type to allocate water
3	trn_obs	int	number of transfer objects that follow

Each transfer-object record is variable-width. The reader reads it twice: first to learn `src_num`, then, after allocating `src(num_src)` and `osrc(num_src)`, to read the source and receiving fields. The full record is:

`id  trn_typ  trn_typ_name  amount  right  src_num  dtbl_src  src(1)..src(src_num)  rcv`

#	Field	Type	Description
1	id	int	transfer object index (also stored as `trn%num`)
2	trn_typ	char(10)	one of `outflo`, `ave_day`, `div_min`, `div_frac`, `dtbl_con`, `dtbl_lum`
3	trn_typ_name	char(40)	name of the dtbl row (for `dtbl_lum` / `dtbl_con`) or recall (for `outflo` with `osrc`); otherwise free text
4	amount	real	demand amount. m3/day for urban objects; mm for HRU; m3/s for `ave_day`, `div_min`, `div_frac`
5	right	char(2)	water right priority. `sr` (senior) or `jr` (junior)
6	src_num	int	number of source objects in this record
7	dtbl_src	char(25)	decision table name to allocate among sources (may be `null`)
8 to 7+8*src_num	src block	source-object tuple, repeated `src_num` times. See below.	
last three	rcv block	receiving-object tuple. See below.	

Source-object tuple (`transfer_source_objects`, 8 columns per source):

#	Field	Type	Description
1	typ	char(10)	source type: `cha`, `res`, `aqu`, `osrc`, `osrc_a`, `wtp`, `use`, `can`
2	num	int	sequential number of the source object in its own database
3	conv_typ	char(10)	conveyance type: `pipe`, `pump`, or none
4	conv_num	int	sequential number of the conveyance object
5	dtbl_lim	char(25)	decision-table name that sets the withdrawal limit (may be `null`)
6	wdraw_lim	real	actual withdrawal limit. Units depend on source type: reservoir uses principal-storage fraction; aquifer uses maximum depth (m); channel uses minimum flow (m3/s)
7	frac	real	fraction of total transfer supplied by this source
8	comp	char(1)	compensate from this source if other sources are past their threshold. `y` or `n`

Receiving-object tuple (`transfer_receiving_objects`, 3 columns):

#	Field	Type	Description
1	typ	char(10)	receiver type: `hru`, `cha`, `res`, `aqu`, `wtp`, `use`, `stor`, `can`, `orcv`
2	num	int	sequential number of the receiver in its own database
3	frac	int	for an HRU receiver, the soil layer that receives incoming tile flow. Declared `integer :: frac = 0.` in the module (initializer is a real literal).

Synthetic example (no example exists in `refdata/`):

```
water_allocation.wro: synthetic example
1
name                      rule_typ          trn_obs
muni_supply               demand            1
id  trn_typ      trn_typ_name      amount    right  src_num  dtbl_src  [src...]
1   ave_day      none              0.10      sr     1        null      cha 5  none 0  null  0.0   1.0   n
```

`TODO: verify` exact whitespace and `null` keyword handling in `dtbl_src`. The reader uses list-directed input, so any whitespace separates fields, but the `null` token convention is inferred from other SWAT+ inputs.

`water_canal.wal`

Reader: src/water_canal_read.f90. Filename is hard-coded in the reader; not pointed to from `file.cio`.

Layout:

- Line 1: title (skipped).
- Line 2: `imax`, the number of canal objects.
- Line 3: header (skipped).
- `imax` canal records, each read twice (back-spaced) to learn `num_aqu` before allocating `aqu_loss(num_aqu)`.

Canal record (`water_canal_data`):

#	Field	Type	Units	Description
1	id	int	none	record index
2	name	char(25)	none	canal name
3	w_sta	char(25)	none	nearby weather station
4	init	char(25)	none	initial concentrations record
5	dtbl	char(25)	none	decision table that determines canal outflow
6	ddown_days	real	days	days to drawdown storage to zero
7	w	real	m	top width
8	d	real	m	depth
9	s	real	m/m	slope
10	ss	real	m/m	side slope (trapezoidal)
11	sat_con	real	varies	parameter used to compute percolation to groundwater
12	loss_fr	real	fraction	water loss during conveyance
13	bed_thick	real	m	bed-sediment thickness for Darcy seepage in `gwflow`; 0 if not used
14	div_id	int	none	recall diversion ID (gwflow). 0 if routed by `wallo`
15	day_beg	int	Julian	canal operation start day (gwflow external)
16	day_end	int	Julian	canal operation end day (gwflow external)
17	num_aqu	int	none	number of aquifers that receive seepage loss
18 .. 17+2*num_aqu	aqu_loss block	int, real pairs		each tuple is `aqu_num` (aquifer index) and `frac` (fraction of loss to that aquifer)

`water_pipe.wal`

Reader: src/water_pipe_read.f90. Filename hard-coded.

Layout:

- Line 1: title (skipped).
- Line 2: `imax`.
- Line 3: header (skipped).
- For each of `imax` pipes:
  - Header line (skipped).
  - One record, read twice (back-spaced) to learn `num_aqu`.

Pipe record (`water_transfer_data`):

#	Field	Type	Units	Description
1	id	int	none	record index
2	name	char(25)	none	pipe name
3	stor_mx	real	m3	maximum storage
4	ddown_days	real	days	drawdown time
5	loss_fr	real	fraction	water loss during conveyance
6	num_aqu	int	none	number of aquifers that receive loss

#	Field	Type	Units	Description
7 .. 6+2*num_aqu	aqu_loss block	int, real	aquifer index and loss fraction	

The `init` field exists in the type definition (`character (len=25) :: init`) but is not read by `water_pipe_read.f90`. `TODO: verify` whether this is by design or an oversight.

`water_tower.wal`

Reader: src/water_tower_read.f90. Filename hard-coded.

Layout:

- Line 1: title (skipped).
- Line 2: `imax`.
- Line 3: header (skipped).
- For each of `imax` towers:
  - Header line (skipped).
  - One record.

Tower record (subset of `water_transfer_data`):

#	Field	Type	Units	Description
1	id	int	none	record index
2	name	char(25)	none	tower name
3	stor_mx	real	m3	maximum storage
4	ddown_days	real	days	drawdown time
5	loss_fr	real	fraction	storage loss

The reader does not allocate or read `aqu_loss` for towers even though the type carries the array. The tower is treated as a passive storage node.

`water_treat.wal`

Reader: src/water_treatment_read.f90. Filename hard-coded.

Layout:

- Line 1: title (skipped).
- Line 2: `imax`.
- Line 3: header (skipped).
- For each of `imax` plants:
  - One main record.
  - If `cs_db%num_pests > 0`: a header line then one record of `num_pests` pesticide concentrations into `wtp_cs_treat(iwtp)%pest`.
  - If `cs_db%num_paths > 0`: a header line then one record of `num_paths` pathogen concentrations into `wtp_cs_treat(iwtp)%path`.

Main record (`water_treatment_use_data`):

#	Field	Type	Units	Description
1	id	int	none	record index
2	name	char(25)	none	plant name
3	stor_mx	real	m3	maximum storage
4	lag_days	real	days	treatment time (outflow lag)
5	loss_fr	real	fraction	water loss during treatment
6	org_min	char(25)	none	name of `om_treat` record for sediment, carbon, nutrients. Cross-walked to `iorg_min`

#	Field	Type	Units	Description
7	pests	char(25)	none	name of pesticide-set record (`ipests`)
8	paths	char(25)	none	name of pathogen-set record (`ipaths`)
9	salts	char(25)	none	name of salt-ion record (`isalts`)
10	constit	char(25)	none	name of other-constituent record (`iconstit`)
11	descrip	char(80)	none	free-text description

`init` is declared in the type but commented out in the source. The reader does not read it. The cross-walk loop sets `iorg_min` only; `ipests`, `ipaths`, `isalts`, `iconstit` are left at 0. `TODO: verify` whether the missing cross-walks are intentional.

`water_use.wal`

Reader: src/water_use_read.f90. Filename hard-coded.

Same `water_treatment_use_data` type and same layout as `water_treat.wal`, including the conditional `pest` and `path` concentration blocks. Records are stored in `wuse` instead of `wtp`. Cross-walk is against `om_use_name` rather than `om_treat_name`.

`out_src.wal`

Reader: src/water_osrc_read.f90. The reader checks `inquire(file='outside_src.wal')` but then opens `'out_src.wal'`. See the Important section.

Layout (as written, assuming the open succeeds):

- Line 1: title (skipped).
- Line 2: `imax`.
- Line 3: header (skipped).
- For each of `imax` sources: one main record, then conditional pest and path blocks (header + values) as in `water_treat.wal`.

Main record (`outside_basin_source`):

#	Field	Type	Units	Description
1	id	int	none	record index
2	name	char(25)	none	out-of-basin source name
3	stor_mx	real	m3	maximum storage
4	lag_days	real	days	lag
5	loss_fr	real	fraction	loss fraction

`outside_rcv.wal`

Reader: src/water_orcv_read.f90. Filename hard-coded. The reader is not called from `main.f90`. See the Important section.

Layout (as written):

- Line 1: title (skipped).
- Line 2: `imax`.
- Line 3: header (skipped).
- For each of `imax` receivers: one record.

Main record (`outside_basin_receive`):

#	Field	Type	Description
1	id	int	record index
2	name	char(25)	out-of-basin receiver name
3	filename	char(25)	name of the output file written for this receiver

#	Field	Type	Description

element.wro and water_rights.wro

Declared in input_file_module.f90 (in_watrts%element, in_watrts%water_rights). No reader in water_*_read.f90 or anywhere else under src/ references either filename. TODO: verify whether these are legacy placeholders or are read by code outside the water-allocation subroutines.

Related

- *.dtl decision tables drive transfers via the water_rights action. dtbl_lum rows are looked up by trn_typ_name from dtbl_lum and supply irrigation amount; dtbl_con rows are looked up from dtbl_flo and supply flow-control amount, source availability and source/receiver allocation. dtbl_src and dtbl_lim are matched to decision-table names at run time.
- object.cnt has a wro column (position 21). It is currently noted in the source as "not used"; the actual count of water allocation objects is taken from line 2 of water_allocation.wro.
- recall_db is cross-walked for osrc sources (daily/monthly/yearly recall) and exco_db for osrc_a (annual constant from an exco record).
- [om_treat.om, om_use.om, om_osrc.om] supply the organic and mineral concentration sets named in org_min fields of water_treat.wal, water_use.wal and out_src.wal.